

Vision-Language-Action 모델

RT2에서 π_0 까지, 그리고 그 너머

Giseop Kim

APRL, DGIST

1. **왜 VLA인가:** 2023년 Coke can 데모가 정말로 의미한 것
2. **Google의 길:** SayCan $\rightarrow$ RT1 $\rightarrow$ PaLM-E $\rightarrow$ RT2
3. π_0 **아키텍처:** PaliGemma + Action Expert
4. **내부 동작:** Attention head가 “pen”을 어떻게 찾는가
5. **Flow Matching:** 이미지 생성과 로봇 제어의 통합
6. **모델 결합 트릭:** KV cache를 통한 LLM-Expert 공유
7. **비판과 대안:** World model 패러다임 (LeCun의 시각)

전제 지식: Transformer, attention, embedding, softmax, diffusion 기초

Part 1. Coke Can과 Taylor Swift

2023년 7월: 어색했지만 결정적인 데모

셋업

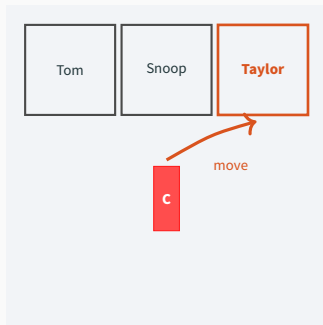
- Google 연구실의 책상
- Coke can 1개
- Tom Cruise, Snoop Dogg, Taylor Swift 사진

명령

“Move the Coke can to Taylor Swift”

실행

- RT2는 너무 커서 로봇에 못 올림
- 카메라 → TPU 클러스터 → 제어 신호
- 천천히, 어색하게 사진 가장자리에 올림



Carol Houseman: “이게 정말 될 거라는
확신이 든 순간이었다.”

왜 이 어색한 데모가 돌파구였나

핵심 관찰: 학습 데이터에 Taylor Swift는 단 한 번도 등장하지 않았다.

모델이 이 명령을 수행하려면, 두 종류의 지식을 **연결**해야 했다:

(A) 인터넷 사전학습

- “Taylor Swift는 사람이다”
- “이 얼굴이 Taylor Swift다”
- 추상적 개념, 의미 관계

(B) 로봇 제어 시연

- “물체 A를 위치 B로”
- gripper 궤적, 힘 제어
- physical grounding

함의

LLM이 **인터넷의 모든 지식**을 **실제 행동**과 연결하는 방법을 학습할 수 있다는 **최초의 정량적 증거**.

이후 1년 내 RT2 핵심 멤버들이 Google을 나와 **Physical Intelligence** 창업.

Part 2. Google의 진화 경로

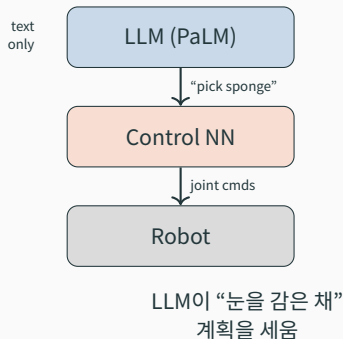
아이디어: LLM이 자연어로 task를 subtask로 분해

예시: “엎질러진 것을 닦아라”

1. 스폰지를 찾는다
2. 스폰지를 집는다
3. 엎질러진 곳으로 간다
4. 닦는다

한계

- 제어는 별도의 imitation network
- LLM은 **미리 정해진 메뉴**에서만 선택
- 새 객체/장면은 재학습 필요



Robot Transformer 1

- Imitation learning, 그러나 **대규모**
- **130,000+** 인간 시연 데이터셋
- Transformer 기반 아키텍처
- 입력: 카메라 이미지 + 언어 지시
- 출력: 로봇 제어 신호 (joint/gripper)

효과: SayCan의 LLM 플래너에 **훨씬 풍부한 메뉴** 제공

- Long-horizon 작업 성능 크게 향상
- 처음 보는 주방에서 특정 물건 찾기 가능

여전한 문제

플래너 LLM은 **여전히 텍스트 전용**. 시각 정보 없이 “눈먼 채” 계획.

2023년 3월: PaLM-E — 시각을 가진 플래너

한 줄 요약: PaLM에 이미지 입력을 직접 통합한 멀티모달 LLM

새로 가능해진 것

- 적응형 플래닝: 원하는 물체에 닿으려 다른 물체를 옮김
- 자율 회복: 누가 칩 봉지를 다시 서랍에 넣어도 즉시 인식하고 재계획

스택의 형태: PaLM-E (플래너) + RT1 (제어)



관찰: 두 모델이 너무 닮았다

PaLM-E

- Vision encoder
- Transformer
- 출력: **텍스트**
- 학습: 다음 토큰 예측, 캡셔닝, task 분해

RT1

- Vision encoder
- Transformer
- 출력: **제어 신호**
- 학습: 인간 시연 모방

자연스러운 질문

PaLM의 출력 vocabulary에 **action token**을 추가해서,
하나의 모델이 텍스트도 행동도 출력하면 어떨까?

즉, LLM이 곧 로봇 두뇌가 될 수 있는가?

레시피

1. PaLM-E와 PaLI-X(또 다른 멀티모달 LLM)에서 출발
2. RT1과 동일한 인간 시연 데이터로 fine-tune
3. 단, 출력을 **토큰화된 제어 신호**로 변환해 학습

결과: 학습 데이터에 없던 객체/장면/지시에 **일반화**

새로운 패러다임의 이름

Vision-Language-Action (VLA)

Vision + Language + Action = 단일 통합 모델

RT2 모델군: 5B ~ 55B 파라미터.

크기 때문에 로봇에서 직접 구동 불가 → 클라우드 추론 필요.

Part 3. π_0 아키텍처 해부

2024년 10월: π_0 — 더 작고, 더 빠르고, 더 잘하는

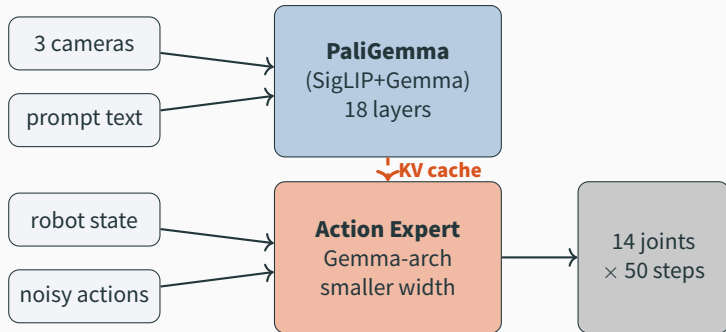
Physical Intelligence가 RT2 대비 보여준 도약

- 빨래 개기, 식탁 정리, 그릴드 치즈, 커피, 오렌지 까기, 자물쇠 열기
- 처음 보는 침실/주방까지 일반화

놀라운 점: 모델이 더 작아졌다

	RT2	π_0
파라미터 수	5B-55B	3.3B
구동 환경	TPU 클러스터	RTX 4090 (on-board)
추론 시간	—	73 ms

질문: 어떻게 작아지면서도 더 잘하게 만들었는가?



핵심 설계 결정 3가지

1. PaliGemma 백본 + 별도 **Action Expert** 모듈
2. Action Expert는 **flow matching**으로 행동 생성
3. 두 모듈은 **KV cache 공유**로 매끄럽게 결합

설계 결정 1: 왜 Action Expert를 따로 두는가

RT2 방식: LLM이 직접 action token 출력

π_0 방식: Gemma는 “인지”에, 별도 expert는 “운동”에 특화

Action Expert의 정체

- Gemma와 **동일한 아키텍처** (실제 코드에서도 Gemma 클래스로 인스턴스화)
- 차이점 1: **무작위 초기화** (사전학습 없음)
- 차이점 2: 각 layer의 **width가 더 작음** (연산량 감소)

SayCan과의 차이 — 매우 중요

SayCan: 두 모듈이 **자연어**로 통신 (저대역폭)

π_0 : 두 모듈이 **attention 내부 표현**으로 통신 (고대역폭)

⇒ 사실상 “하나의 두뇌처럼” 사고하면서도 모듈성을 유지

Part 4. 내부 동작: Attention head

Gemma LLM이 받는 입력

이미지 처리

- 카메라 3대 (overhead + 양쪽 wrist)
- 각 이미지 → 256 patch → 총 **768 patches**
- Image encoder → 각 patch당 길이 2048짜리 **embedding vector**

텍스트 처리

- “uncap the pen” → 4 토큰 → 4 임베딩 벡터

총합: $768 + 4 = 772$ soft tokens

Gemma 구조

- 18 transformer blocks
- 각 block: attention (8 heads) + MLP

Attention head 한 개의 연산

입력: 772개의 임베딩 벡터 행렬 $X \in \mathbb{R}^{772 \times d}$

세 행렬 생성

$$Q = XW_Q, \quad K = XW_K, \quad V = XW_V$$

각각 772개의 행 $\Rightarrow$ 입력 토큰 하나당 query/key/value 한 개씩.

Attention 패턴

$$A = \text{softmax}\left(\frac{QK^T}{\sqrt{d_k}}\right) \in \mathbb{R}^{772 \times 772}$$

출력

$$\text{head}(X) = AV \in \mathbb{R}^{772 \times d_v}$$

직관: 각 query는 “나에게 관련된 정보가 어디 있나?”를 묻고,
AV 곱은 “그 위치의 value를 내 자리에 가져온다”를 수행.

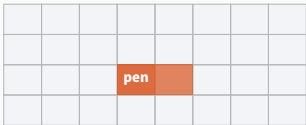
사례 연구: “pen” 토큰이 펜을 찾는다

시나리오: 프롬프트의 마지막 토큰 “pen”의 query 벡터 q_{pen}

관찰된 행동

1. q_{pen} 과 768개 image patch key 사이 dot product
2. 압도적으로 큰 값이 **펜이 보이는 두 patch**에서 발생
3. Softmax 후 attention map을 이미지에 입히면 **펜만 빛난다**

Overhead camera — attention map



⇒ 모든 프레임에서 **펜을 자동 추적**. 이것은 18 layer × 8 head 중 **단 한 head**의 동작.

Attention head의 의미론적 해석

AV 곱이 실제로 하는 일

큰 attention 값 a_{ij} 가 있을 때:

$$\text{output}_i += a_{ij} \cdot v_j$$

즉, “pen” 토큰 위치 i 에 **펜 patch** 위치 j 의 value 벡터가 복사되어 더해짐.

한 줄 해석

이 attention head는 텍스트의 “pen”과 이미지 속 펜을 **하나의 통합 표현**으로 묶는다.

전체 그림

- 18 layer $\times$ 8 head = **144개의 attention head**
- 각 head가 다른 종류의 패턴을 학습
- 객체 위치, 손-물체 관계, 시간적 일관성, 작업 진행도 등

Part 5. Action Expert와 Flow Matching

Action Expert의 입력

입력 1: 로봇 상태 (1 token)

- Aloha 플랫폼: 팔당 7 joint $\times 2 =$ **14차원 벡터**
- Waist, shoulder, elbow, forearm rot, wrist, wrist rot, gripper
- 14×1024 가중치 행렬로 길이 **1024** embedding으로 매핑

입력 2: 노이즈가 섞인 미래 행동 (50 tokens)

- 다음 50 시간 단계의 14차원 joint 위치 예측
- 초기에는 순수 노이즈로 시작

총 51 tokens (Gemma의 2048 대비 **1024**로 축소된 차원 사용)

⇒ Action expert는 가벼움. 그래서 여러 번 반복 실행 가능.

Flow Matching: 노이즈에서 궤적으로

직관: 이미지 생성에서 “노이즈 $\rightarrow$ 고양이 사진”하는 것처럼,
“노이즈 $\rightarrow$ 14×50 관절 궤적”을 만든다.

왜 잘 작동하는가? 분포가 **multimodal**이기 때문

- 고양이를 그리는 방법은 여러 가지
- 펜 뚜껑을 여는 방법도 여러 가지 (어느 손, 어느 각도, 어느 속도)
- Single-mode regression은 평균(mode collapse)을 학습 $\rightarrow$ 어색한 동작

π_0 의 절차

1. 무작위 14×50 행렬 $a^{(0)}$ 생성
2. Action expert로 update Δa 계산
3. $a^{(t+1)} = a^{(t)} + \Delta a$
4. **10번 반복** $\rightarrow$ 매끄러운 최종 궤적

이미지 생성과 로봇 제어의 통합

이미지 생성

- 입력: 텍스트 + 노이즈 이미지
- 출력: 자연 이미지
- 분포: 자연 이미지 manifold

noise $\rightarrow$ cat

로봇 제어 (π_0)

- 입력: 프롬프트 + 이미지 + 노이즈 궤적
- 출력: 14×50 joint 궤적
- 분포: 가능한 동작 manifold

noise $\rightarrow$ trajectory

매우 강력한 추상화

완전히 다른 응용처럼 보이는 두 영역에서 **같은 수학적 프레임워크**가 작동한다는 것은 우연이 아닌 **심오한 통일성**.

참고: 이 통찰이 Diffusion Policy, RDT, Octo 등 후속 연구의 토대가 됨.

Part 6. 두 모듈의 결합 — KV cache 트릭

Action expert도 $18 \text{ layer} \times 8 \text{ head}$ 를 가진다.

Action expert의 query는 무엇을 “보고” 싶은가?

- 다른 시간 단계의 행동들 (부드러운 궤적을 위해)
- 프롬프트 (“무엇을 해야 하는가”)
- 이미지 (“펜이 어디 있는가”)
- 로봇 상태 (“지금 어디 있는가”)

문제: 프롬프트와 이미지 정보는 PaliGemma의 attention 안에 있다.
어떻게 action expert가 거기에 접근하게 할까?

해법: Key/Value를 이어 붙이기

같은 아키텍처 $\Rightarrow$ 같은 layer index에서 호환

Action expert의 layer ℓ 에서 attention 계산할 때:

$$K_{\text{total}} = \begin{bmatrix} K_{\ell}^{\text{AE}} \\ K_{\ell}^{\text{Gemma}} \end{bmatrix}, \quad V_{\text{total}} = \begin{bmatrix} V_{\ell}^{\text{AE}} \\ V_{\ell}^{\text{Gemma}} \end{bmatrix}$$

크기: $51 + 772 = 823$ keys/values

Action expert의 query는 자기 51 token (시간적 일관성)과 Gemma 772 token (의미/시각)을 한 번의 attention으로 **동시에** 참조 가능.

왜 이게 가능한가

같은 width, 같은 layer 수, 같은 head 차원 $\Rightarrow$ key/value 공간의 **차원 호환성**.

효율성: KV cache 재사용

표준 LLM 추론에서의 KV cache

- 새 토큰이 들어올 때마다 이전 K, V 재계산하지 않고 캐시

π_0 가 영리하게 활용

1. PaliGemma forward pass 1번 → 모든 layer의 $K_{\ell}^{\text{Gemma}}, V_{\ell}^{\text{Gemma}}$ 캐시
2. Flow matching은 **10번 반복** 필요
3. 매 반복마다 **같은 캐시 재사용** (이미지/프롬프트는 안 바뀜)

성능 영향

이 캐싱 덕분에 **10회 디노이징을 73 ms**에 끝낼 수 있음.
⇒ on-board RTX 4090에서 실시간 제어 가능.

1. 카메라 3장 + 텍스트 프롬프트 + 현재 robot state 수집
2. PaliGemma forward pass → 모든 layer의 K/V 캐시
3. Action expert 초기화: state token + 50개 noise token
4. **Flow matching 루프 (10회)**
 - 각 layer에서 self-attention + cross-attention via concatenated K/V
 - Δa 계산, 궤적 업데이트
5. 최종 14×50 궤적 출력
6. 로봇이 처음 몇 step 실행
7. 새 이미지 받아 1번부터 반복

73 ms / cycle → 약 13 Hz 제어 루프

Part 7. 비판적 시각과 미래

데모를 신중히 읽기: 1995년 Ralph

Carnegie Mellon, 1995

- 자율주행 시스템 “Ralph”
- 미국 횡단 **98.2% 자율**

30년 후의 자율주행

- Ralph의 직계 후손이 아니다
- 센서 융합, HD map, learned planner, end-to-end transformer 등 **전혀 다른 패러다임**

교훈

인상적인 데모와 “실제로 작동하는 시스템”은 **다른 문제**.

π_0 도 “집안일을 다 해주는 로봇”까지는 한참 남았다.

대안 패러다임: World Models

등장 배경: VLA가 정말 “올바른 길”인지에 대한 의문

Yann LeCun (전 Meta 수석 AI 과학자)

- Meta를 떠나 **world model 중심** 벤처 창업
- LLM 백본을 쓰지 **않는** 방향, JEPA 계열

LeCun의 입장 (인용)

“VLA are doomed. They basically don’t work really well.”

핵심 차이

- VLA: 인터넷 텍스트/이미지 사전학습 → 행동으로 fine-tune
- World model: 환경의 **예측 모델**을 latent space에서 학습
- Planning은 학습된 world model 위에서 search/optimization

1. RT2의 일반화 (Taylor Swift)는 진짜 “이해”인가, 아니면 통계적 보간인가?
2. Action expert를 따로 두는 것은 **귀납적 편향**을 주입한 것이다. 이는 “순수 end-to-end” 철학과 충돌하는가, 보완하는가?
3. Flow matching이 mode collapse를 피한다고 했다. 그럼 **원치 않는 mode** (안전하지 않은 동작)도 학습될 위험은?
4. KV cache 공유는 “같은 아키텍처”에 의존한다. **이종 모듈**을 결합하는 더 일반적인 방법은?
5. LeCun의 주장이 옳다면, classical SLAM/state estimation의 역할은 미래 로봇 두뇌에서 어떻게 자리잡을까?
(Hint: APRL의 “Classical-to-Modern Bridge” 관점)

1. **VLA**: Vision + Language + Action을 단일 모델에서 통합

2. π_0 설계 3원칙

- 백본 LLM (PaliGemma) + 별도 Action Expert
- Flow matching으로 궤적 생성
- KV cache 공유로 효율적 모듈 결합

3. 이중 통합

- 인터넷 지식 $\leftrightarrow$ 물리적 행동
- 이미지 생성 수학 $\leftrightarrow$ 로봇 제어 수학

4. 한계와 대안: World model 패러다임은 다른 길을 제시

5. 연구자에게: 데모와 실제 시스템 사이 간극을 잊지 말 것

참고 문헌 (시작점)

- Ahn et al. (2022) “Do As I Can, Not As I Say: Grounding Language in Robotic Affordances” (SayCan)
- Brohan et al. (2022) “RT-1: Robotics Transformer for Real-World Control at Scale”
- Driess et al. (2023) “PaLM-E: An Embodied Multimodal Language Model”
- Brohan et al. (2023) “RT-2: Vision-Language-Action Models Transfer Web Knowledge to Robotic Control”
- Black et al. (2024) “ π_0 : A Vision-Language-Action Flow Model for General Robot Control”
- Beyer et al. (2024) “PaliGemma: A versatile 3B VLM for transfer”
- Lipman et al. (2023) “Flow Matching for Generative Modeling”
- LeCun (2022) “A Path Towards Autonomous Machine Intelligence” (JEPA position paper)

다음 주: World model과 JEPA 계열 깊이 다루기